AI Assissted Coding

LAB ASSIGNMENT – 13

Name : G.Abhiram

HT.No : 2303A51087

Batch No : 02

Task – 01

Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

• Instructions:

- Prompt AI to identify duplicate logic and replace it with functions or classes.
- Ensure the refactored code maintains the same output.
- Add docstrings to all functions.

• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
```

print("Perimeter of Rectangle:", 2 * (10 + 15))

- Expected Output:
  - Refactored code with a reusable function and no duplication.
  - Well documented code

Code:

```
# refactor the code to calcuate area and perimeter by removing code duplication
'''
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
'''

def calculate_area_perimeter(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
lengths = [5, 7, 10]
widths = [10, 12, 15]
for length, width in zip(lengths, widths):
    area, perimeter = calculate_area_perimeter(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
```

Output:

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task – 02

Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

- Identify repeated or related logic and extract it into reusable functions.
- Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

Week7

-

Monda

y

Legacy script with inline repeated logic

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

- **Expected Output:**
  - Code with a function calculate_total(price) that can be reused

for multiple price inputs.

  - Well documented code

Code :

```
# refactor the code to calculate total price by using funcation and for multiple prices
'''
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
'''

def calculate_total_price(price):
    tax = price * 0.18 # Calculate tax as 18% of the price
    total = price + tax # Calculate total price by adding tax to the original price
    return total
prices = list(map(int,input().split())) # List of prices to calculate total price for
for price in prices:
    total_price = calculate_total_price(price) # Call the function to calculate total price for each price in the list
    print("Total Price:", total_price)
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/a
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
250 345 497 543
Total Price: 295.0
Total Price: 407.1
Total Price: 586.46
Total Price: 640.74
```

Task – 03

Problem Statement

The given script contains duplicated conditional statements used to assign

grades based on student marks. This redundancy violates clean code

principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve

modularity, reusability, and readability while preserving the original grading

logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator

2. Method Name: calculate_grade(self, marks)

3. The method must:

- Accept marks as a parameter.
- Return the corresponding grade as a string.
- The grading logic must strictly follow the conditions below:

▪ Marks ≥ 90 and ≤ 100 → "Grade A"

▪ Marks ≥ 80 → "Grade B"

▪ Marks ≥ 70 → "Grade C"

▪ Marks ≥ 40 → "Grade D"

▪ Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

- The class
- The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")
```

```
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

Code :

```python
# refactor the code by using class and function to calculate grade based on marks and reusability of code
'''
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
'''
class GradeCalculator:
    def calculate_grade(self, marks):
        '''
        Calculate the grade based on the marks.
        Args:
            marks (int): The marks obtained by the student.
        Returns:
            str: The grade corresponding to the marks.
        '''
        if marks >= 90:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"

        else:
            return "Fail"
grade_calculator = GradeCalculator()
marks_list = list(map(int, input("Enter marks separated by space: ").split())) # List of marks to calculate grades for s
for marks in marks_list:
    grade = grade_calculator.calculate_grade(marks) # Call the method to calculate grade for each marks in the list
    print(f"Marks: {marks}, Grade: {grade}")
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/abhir/OneDr
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter marks separated by space: 56 89 32 99 76 0 54
Marks: 56, Grade: Grade D
Marks: 89, Grade: Grade B
Marks: 89, Grade: Grade B
Marks: 32, Grade: Fail
Marks: 32, Grade: Fail
Marks: 99, Grade: Grade A
Marks: 76, Grade: Grade C
Marks: 0, Grade: Fail
Marks: 54, Grade: Grade D
```

Task – 04

Use AI to refactor procedural input–processing logic into

functions.

Instructions:

- Identify input, processing, and output sections.
- Convert each into a separate function.
- Improve code readability without changing behavior.

• Sample Legacy Code:

```
num = int(input("Enter number: "))

square = num * num

print("Square:", square)
```

• Expected Output:

- Modular code using functions like get_input(), calculate_square(), and

display_result().

Code :

```python
lab-13.py > ...
# refactor the code using function and add functions to get user input and calculate square of a number an displa
'''num = int(input("Enter number: "))
square = num * num
print("Square:", square)'''

def get_user_input():
    num = int(input("Enter number: "))
    return num
def calculate_square(num):
    return num * num
number = get_user_input() # Get user input
square = calculate_square(number) # Calculate square of the number
print("Square:", square) # Display the result
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/User
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter number: 65
Square: 4225
```

Task – 05 :

Use AI to refactor procedural code into a class-based design.

Focus Areas:

- Object-Oriented principles
- Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

- A class like EmployeeSalaryCalculator with methods and attributes.

Code :

```
# refactor the code using class and function to calculate net salary after tax deduction with methods and attributes
'''salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)'''

class SalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        net_salary = self.salary - tax
        return net_salary
salary = int(input("Enter your salary: ")) # Get user input for salary
calculator = SalaryCalculator(salary) # Create an instance of SalaryCalculator with the given salary
net_salary = calculator.calculate_net_salary() # Calculate net salary using the method
print("Net Salary:", net_salary) # Display the result
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/User
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter your salary: 45000
Net Salary: 36000.0
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/User
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter your salary: 98000
Net Salary: 78400.0
```

Task – 06

Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

- Time complexity
- Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- Use of sets or dictionaries with complexity justification

Code :

```
lab-13.py > ...
# refactor the code uisng set or dictionaries and with low  time complexity datastructure
'''users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")'''

users = {"admin", "guest", "editor", "viewer"} # Using a set for O(1) average time complexity for lookups
name = input("Enter username: ")
if name in users: # Check if the username is in the set
    print("Access Granted")
else:
    print("Access Denied")
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/a
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter username: admin
Access Granted
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/a
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter username: abhiram
Access Denied
```

Task – 07

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation

• Uses print-based procedural execution

• Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

- add_book(title, author, isbn)
- remove_book(isbn)
- search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)

# This code needs refactoring into a proper module with documentation.

library_db = {}

# Adding first book

title = "Python Basics"

author = "John Doe"

isbn = "101"

if isbn not in library_db:
```

```
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
```

```
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
else:
print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
```

Code :

library.py > search_book

```python
# refactor the code use modules for add ,search and remove books from library database and avoid code duplication using
def add_book(library_db, title, author, isbn):
    '''
    Adds a book to the library database if it doesn't already exist.
    Parameters:
    library_db (dict): The library database to which the book will be added.
    title (str): The title of the book.
    author (str): The author of the book.
    isbn (str): The ISBN number of the book, used as a unique identifier.
    '''
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        print("Book added successfully.")
    else:
        print("Book already exists.")
def search_book(library_db, isbn):
    '''
    Searches for a book in the library database using its ISBN number.
    Parameters:
    library_db (dict): The library database to search through.
    isbn (str): The ISBN number of the book to search for.
    '''
    if isbn in library_db:
        print("Book Found:", library_db[isbn])
    else:
        print("Book not found.")
def remove_book(library_db, isbn):
    '''
    Removes a book from the library database using its ISBN number.
    Parameters:
        library_db (dict): The library database from which the book will be removed.
        isbn (str): The ISBN number of the book to remove.
        '''
    if isbn in library_db:
        del library_db[isbn]
        print("Book removed successfully.")
    else:
        print("Book not found.")
library_db = {}
add_book(library_db, "The Great Gatsby", "F. Scott Fitzgerald", "978-0743273565")
search_book(library_db, "978-0743273565")
remove_book(library_db, "978-0743273565")
```

Output :

127.0.0.1:5500/library.html

newtab | Dashboard | SR University | Welcome To Colab -... | GitHub | GDB online Debugg... | EduSkills | Internships | AWS Academy Clou... | TIMETABLE | Abh

index

library c:\users\abhir\onedrive\pictures\documents\cse\3-2\ai-ac\library.py

refactor the code use modules for add ,search and remove books from library database and avoid code duplication using pydoc strings to generate documentation for the function

Functions

add_book(library_db, title, author, isbn)
Adds a book to the library database if it doesn't already exist.
Parameters:
library_db (dict): The library database to which the book will be added.
title (str): The title of the book.
author (str): The author of the book.
isbn (str): The ISBN number of the book, used as a unique identifier.

remove_book(library_db, isbn)
Removes a book from the library database using its ISBN number.
Parameters:
library_db (dict): The library database from which the book will be removed.
isbn (str): The ISBN number of the book to remove.

search_book(library_db, isbn)
Searches for a book in the library database using its ISBN number.
Parameters:
library_db (dict): The library database to search through.
isbn (str): The ISBN number of the book to search for.

Data
library_db = {}

Task – 08

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
a=b
b=c
```

Code :

```python
# refactor the code to clean resuable function to calculate fibonacci series up to n terms and display the result

def fibonacci(n):
    """ this function calculates the fibonacci series up to n terms and returns the result as a list
    n: the number of terms to calculate in the fibonacci series
    returns: a list containing the fibonacci series up to n terms
    """
    fib_series = []
    a, b = 0, 1
    for _ in range(n):
        fib_series.append(a)
        a, b = b, a + b
    return fib_series
n = int(input("Enter limit: "))
print(fibonacci(n))

# test cases
#print(fibonacci(10))  # Expected output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
#print(fibonacci(0))   # Expected output: []
#print(fibonacci(1))   # Expected output: [0]
#print(fibonacci(2))   # Expected output: [0, 1]
```

Output :

```
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter limit: 25
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368]
```

Task – 09

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).

• Add docstrings and optimize.

• Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")
```

Code :

```python
# refactor the code to check if two numbers are twin primes by using a function and with low time complexity

def is_prime(num):
    """ this function checks if a number is prime or not and returns the result as a boolean
    num: the number to be checked
    returns: True if num is prime, False otherwise
    """
    if num <= 1:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True
def are_twin_primes(a, b):
    """ this function checks if two numbers are twin primes or not and returns the result as a boolean
    a: the first number to be checked
    b: the second number to be checked
    returns: True if a and b are twin primes, False otherwise
    """
    if is_prime(a) and is_prime(b) and abs(a - b) == 2:
        return True
    return False
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if are_twin_primes(a, b):
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

Output :

```
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter first number: 10
Enter second number: 45
Not Twin Primes
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/al
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter first number: 5
Enter second number: 7
Twin Primes
```

Task -10

Refactor the given poorly structured Python script into a clean, modular, and

reusable implementation.

The current program reads a year from the user and prints the corresponding

Chinese Zodiac sign. However, the implementation contains repetitive

conditional logic, lacks modular design, and does not follow clean coding

principles.

Your task is to refactor the code to improve readability, maintainability, and

structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat

2. Ox

3. Tiger

4. Rabbit

5. Dragon

6. Snake

7. Horse

8. Goat (Sheep)

9. Monkey

10. Rooster

11. Dog

12. Pig

```
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")
```

```
elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

Code :

```python
# refactor the code using function to get zodiac and replace the if else statements with a dictionary for better readab


def get_zodiac(year):
    """ this function returns the zodiac sign for a given year based on the Chinese zodiac system
    year: the year to get the zodiac sign for
    returns: the zodiac sign corresponding to the given year
    """
    zodiacs = {
        0: "Monkey",
        1: "Rooster",
        2: "Dog",
        3: "Pig",
        4: "Rat",
        5: "Ox",
        6: "Tiger",
        7: "Rabbit",
        8: "Dragon",
        9: "Snake",
        10: "Horse",
        11: "Goat"
    }
    return zodiacs[year % 12]
year = int(input("Enter a year: "))
print(get_zodiac(year))
```

Output :

```
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a year: 3
Pig
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/ab
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a year: 7
Rabbit
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/ab
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a year: 9
Snake
```

Task – 11

Refactor the given poorly structured Python script into a clean, modular, and

reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 (Harshad Number)
- 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10
```

if sum_digits != 0:

if num % sum_digits == 0:

print("True")

else:

print("False")

else:

print("False")

You must:

1. Create a reusable function: is_harshad(number)

2. The function must:

- Accept an integer parameter.
- Return True if the number is divisible by the sum of its digits.
- Return False otherwise.

3. Separate user input from core logic.

4. Add proper docstrings.

5. Improve readability and maintainability.

6. Ensure the program handles edge cases (e.g., 0, negative numbers).

Code :

```python
# refactor the code with reusable function user input and handle edge cases to check if a number is a Harshad numb


def is_harshad_number(num):
    """ this function checks if a number is a Harshad number or not and returns the result as a boolean
    num: the number to be checked
    returns: True if num is a Harshad number, False otherwise
    """
    temp = num
    sum_digits = 0
    while temp > 0:
        digit = temp % 10
        sum_digits += digit
        temp //= 10
    if sum_digits == 0:
        return False
    return num % sum_digits == 0
number = int(input("Enter a number: "))
if is_harshad_number(number):
    print("True")
else:
    print("False")
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/ab
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a number: 18
True
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/ab
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a number: 19
False
```

Task – 12

Refactor the given poorly structured Python script into a clean, modular, and

efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
count = count + 1
fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)

2. The function must:

   - Accept a non-negative integer n.
   - Return the number of trailing zeros in n!.

3. Do NOT compute the full factorial.

4. Use an optimized mathematical approach (count multiples of 5).

5. Add proper docstrings.

6. Separate user interaction from core logic.

7. Handle edge cases (e.g., negative numbers, zero).

Code :

```python
# refactor the code with optimized approach and seperate user interaction from core logic ,n should be able to handle l


def count_trailing_zeros(n):
    """ this function counts the number of trailing zeros in the factorial of a given number n
    n: the number to calculate the factorial of
    returns: the number of trailing zeros in n!
    """
    count = 0
    power_of_5 = 5
    while n >= power_of_5:
        count += n // power_of_5
        power_of_5 *= 5
    return count
n = int(input("Enter a number: "))
if n > 0:
    print(count_trailing_zeros(n))
else:
    print("Please enter a positive integer.")
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/User
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a number: 14
2
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/User
python.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a number: 24
```

Task – 13

Function: Generate Collatz sequence until reaching 1.

• Test Cases to Design:

• Normal: 6 → [6,3,10,5,16,8,4,2,1]

• Edge: 1 → [1]

• Negative: -5

• Large: 27 (well-known long sequence)

• Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

• Takes an integer n as input.

• Generates the Collatz sequence (also called the 3n+1 sequence).

• The rules are:

o If n is even → next = n / 2.

o If n is odd → next = 3n + 1.

- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even → 6/2 = 3)
- 3 (odd → 3*3+1 = 10)
- 10 (even → 10/2 = 5)
- 5 (odd → 3*5+1 = 16)
- 16 (even → 16/2 = 8)
- 8 (even → 8/2 = 4)
- 4 (even → 4/2 = 2)
- 2 (even → 2/2 = 1)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

Code :

```python
# refactor the code and generate test cases and check the collatz sequence for a given number n and display the result as a



def collatz_sequence(n):
    """ this function generates the collatz sequence for a given number n and returns the result as a list of numbers in th
    n: the number to generate the collatz sequence for
    returns: a list containing the collatz sequence starting from n and ending at 1
    """
    if n<0:
        raise ValueError("Input must be a non-negative integer")
    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
    sequence.append(1)  # Append the last element of the sequence, which is always 1
    return sequence
n = int(input("Enter a number: "))
print(collatz_sequence(n))
# testcases
#print(collatz_sequence(6))  # Expected output: [6, 3,
# 10, 5, 16, 8, 4, 2, 1]
#print(collatz_sequence(1))  # Expected output: [1]
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/a
hon.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a number: 6
[6, 3, 10, 5, 16, 8, 4, 2, 1]
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/a
hon.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a number: 8
[8, 4, 2, 1]
```

Task – 14

Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then Fn = Fn-1 + Fn-2)

- Test Cases to Design:

- Normal: 5 → [2, 1, 3, 4, 7]
- Edge: 1 → [2]
- Negative: -5 → Error
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

Code :

```
# refactor the code and generate test cases using pytest to generate Lucas sequence up to n terms and display the

def lucas_sequence(n):
    """ this function generates the lucas sequence up to n terms and returns the result as a
    list of numbers in the sequence
    n: the number of terms to generate in the lucas sequence
    returns: a list containing the lucas sequence up to n terms
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer")
    sequence = []
    a, b = 2, 1
    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence
n = int(input("Enter number of terms: "))
print(lucas_sequence(n))
# test cases
#print(lucas_sequence(5))  # Expected output: [2, 1,
# 3, 4, 7]
#print(lucas_sequence(1))  # Expected output: [2]
#print(lucas_sequence(-5))  # Expected output: ValueError
#print(lucas_sequence(10))  # Expected output: [2, 1,
# 3, 4, 7, 11, 18, 29, 47, 76]
```

Output :

```
Enter number of terms: 5
[2, 1, 3, 4, 7]
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/abhir/OneDrive/Pictures/Documents
hon.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter number of terms: -1
Traceback (most recent call last):
  File "c:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC\lab-13.py", line 283, in <module>
    print(lucas_sequence(n))
          ~~~~~~~~~~~~~~^^^
  File "c:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC\lab-13.py", line 275, in lucas_sequence
    raise ValueError("Input must be a non-negative integer")
ValueError: Input must be a non-negative integer
```

Task – 15

Function: Count vowels and consonants in string.

- Test Cases to Design:
- Normal: "hello" → (2,3)
- Edge: "" → (0,0)
- Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

Code :

```python
# refactor the code to countvowels and consonants in a string by using a function and with low time complexity and handle e

from pydoc import text
def count_vowels_consonants(s):
    """ this function counts the number of vowels and consonants in a given string s and returns the result as a tuple of c
    s: the string to count vowels and consonants in
    returns: a tuple containing the count of vowels and consonants in the string (vowels, consonants)
    """
    if not isinstance(s, str):
        raise ValueError("Input must be a string")
    vowels = set("aeiouAEIOU")
    vowel_count = 0
    consonant_count = 0
    for char in s:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count
text = input("Enter a string: ")
vowels, consonants = count_vowels_consonants(text)
print(f"Vowels: {vowels}, Consonants: {consonants}")
# test cases
#print(count_vowels_consonants("Hello World!"))  # Expected output: (3, 7)
#print(count_vowels_consonants("AEIOUaeiou"))  # Expected output: (10, 0)
```

Output :

```
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/
hon.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a string: Abhiram
Vowels: 3, Consonants: 4
(.venv) PS C:\Users\abhir\OneDrive\Pictures\Documents\CSE\3-2\AI-AC> & C:/Users/
hon.exe c:/Users/abhir/OneDrive/Pictures/Documents/CSE/3-2/AI-AC/lab-13.py
Enter a string: Pallavi
Vowels: 3, Consonants: 4
```